

PROMPT IA 16

Production Readiness + PRA + GO / NO-GO

Multi-site validation - performance - resilience - restore - security - capacity - operational readiness

Mission : certifier techniquement la plateforme e-commerce avant ouverture de production, a partir de preuves mesurables et reproductibles, puis produire une decision GO / CONDITIONAL GO / NO-GO.

evidence first - no assumptions - measurable SLO/RTO/RPO - tested recovery

1. Role de l'IA

Tu es Principal SRE / Production Readiness Engineer. Tu dois orchestrer, exécuter et analyser la certification finale de la plateforme. Tu ne dois pas déclarer la production prête sur la base de documents seuls : chaque capacité critique doit être démontrable par un test, une mesure ou une preuve d'audit.

Un composant 'healthy' n'est pas automatiquement production-ready. La certification exige performance, résilience, sécurité, restauration, observabilité, runbooks, ownership et capacité de rollback/failover.

2. Perimetre de certification

Domaine	Ce qui doit être certifié
Application	Parcours Storefront/Admin, backend authoritative, erreurs et modes dégradés.
Performance	RPS, latence, saturation, queue/consumer lag, Core Web Vitals.
Data	PostgreSQL, Kafka, RabbitMQ, Redis, OpenSearch, ClickHouse, reconciliation.
Platform	Kubernetes, Cilium, Istio, FluxCD, Harbor, OpenBao, Keycloak, operators.
Security	IAM, secrets, egress, supply-chain, runtime, audit, KMS/encryption.
Recovery	Backups, PITR, etcd, Git, OpenBao, failover/failback, site rebuild.
Operations	Alerting, runbooks, on-call, incident process, scorecards, ownership.

3. Pre-conditions avant certification

- Prompts 01 à 15 implémentés ou suffisamment complets pour le périmètre testé.
- Images OCI signées, SBOM disponibles et déploiements par digest.
- GitOps réconcilie, drift = 0 sur les composants critiques.
- SLO et alertes actives.
- Backups récents et restore procédures disponibles.
- Runbooks critiques présents dans Backstage.
- Design Packs et parcours UI critiques implémentés.
- Environnement de certification suffisamment proche de la production.
- Jeux de données synthétiques représentatifs et reproductibles.

4. Matrice GO / NO-GO

GO

- > aucun blocker critique
- > SLO/performance atteints
- > restores/failover passés
- > security gates passés
- > runbooks/ownership complets

CONDITIONAL GO

- > aucun risque critique non maîtrisé
- > quelques écarts documentés
- > compensating controls + owner + deadline

NO-GO

- > failover non fiable
- > restore non prouvé
- > split-brain possible

-> security blocker
-> SLO critique non atteint
-> corruption/integrite data non maitrisee

5. Parcours fonctionnels critiques

Executer des tests E2E Playwright et API sur les parcours majeurs.

Storefront:

Home -> Search -> Product -> Cart -> Login -> Checkout
-> Pricing/Tax/Inventory/Fraud/Payment
-> Order CONFIRMED
-> Shipping -> Tracking -> Billing -> Notification
-> Return -> Refund

Admin:

Login -> RBAC -> Product/Price/Inventory
-> Order management -> Refund -> Return -> Fraud -> Audit

Verifier les chemins positifs et negatifs : stock insuffisant, paiement refuse, timeout provider, webhook duplique/perdu, fraude deny, refund retry, carrier down.

6. SLO et latence

Metric	Target
Public API availability	99.9 %
Order/Payment availability	99.95 %
REST p95	< 300 ms
REST p99	< 1 s
gRPC p95	< 100 ms
gRPC p99	< 300 ms
Search p95	< 250 ms
Kafka consumer lag normal	< 30 s

7. Charge et performance k6

REST:

500 RPS nominal
2 000 RPS peak

gRPC:

1 000 req/s nominal
5 000 req/s peak

Kafka:

1 000 events/s nominal
5 000 events/s peak
10 000 events/s burst

Tester ramp-up, steady state, peak, burst, recovery et long-running soak. Capturer CPU, RAM, I/O, DB latency, queue depth, consumer lag, error rate et saturation.

8. Capacity validation

Verifier que le cluster cible 3 control-plane + 5 workers/site supporte la charge avec marge. Produire un capacity headroom explicite.

```
capacity_headroom = available_capacity - peak_observed_usage
```

Verifier:

```
CPU headroom
RAM headroom
NVMe IOPS headroom
network headroom
PostgreSQL connections
Kafka partitions/throughput
RabbitMQ queue pressure
OpenSearch heap/disk
Ceph capacity
object storage growth
```

Ne pas certifier un systeme qui atteint deja 90-100 % de saturation sous le peak attendu.

9. Autoscaling validation

Tester HPA, KEDA et, si applicable, Cluster Autoscaler. Verifier temps de reaction, stabilisation et absence d'oscillation.

Tester VPA recommendations mais ne pas autoriser d'ajustement automatique Tier 0 sans policy explicite.

10. Frontend performance

Executer Lighthouse CI et mesures navigateur sur Storefront NOMA : LCP, INP, CLS, TTFB, bundle JS/CSS, image loading, cache ATS, RSC/SSR et streaming.

Le budget JavaScript est mesure et justifie par l'UX ; refuser les regressions non justifiees.

11. Chaos engineering

- Kill d'un pod critique.
- Perte d'un worker.
- Perte d'un control-plane si testable sans risque hors perimetre.
- Perte d'un broker Kafka.
- Failover PostgreSQL CloudNativePG.
- Redis node failure.
- RabbitMQ node failure / quorum queue continuity.
- OpenSearch node failure.
- Coupure WAN inter-site.
- Latence/perte reseau.
- Provider PSP/carrier indisponible.
- Rollback Flagger.
- Perte complete d'un site actif.

12. Critere de chaos

Chaque experience doit declarer hypothesis, blast radius, abort condition, metric de succes, runbook et temps de recovery observe.

```
experiment_id
hypothesis
start_time
fault
```

```
observed_impact
SLO_breach
recovery_action
RTO_observed
result PASS/FAIL
```

13. Failover multi-site

Tester Site A -> Site B en respectant home_site, fencing/quorum et prevention split-brain.

```
Site A unhealthy
-> detect
-> fence writer ownership
-> promote Site B when safe
-> verify PostgreSQL/Kafka/Redis/app state
-> GSLB traffic shift
-> synthetic business tests
-> SLO monitoring
```

Le DNS/GSLB n'accorde jamais le droit d'ecriture. Le transfert d'ownership/fencing doit etre valide avant ouverture du trafic d'ecriture.

14. Failback

```
Site A rebuilt
-> keep out of traffic
-> resync data
-> reconciliation PASS
-> shadow/synthetic tests
-> transfer ownership explicitly
-> verify fencing
-> progressive traffic return
```

Un site reconstruit ne recupere jamais automatiquement le droit d'ecriture parce qu'il est redevenu joignable.

15. PRA-as-Code - rebuild from zero

Executer une reconstruction automatisee d'un site detruit.

```
1 Infrastructure
2 Kubernetes
3 CNI/DNS
4 OpenBao/PKI bootstrap
5 FluxCD
6 Storage/operators
7 Data restore
8 Platform services
9 Applications
10 Synthetic validation
11 Traffic enablement
```

Mesurer RTO et RPO reels. Conserver logs, captures, checksums, timings et resultat des validations.

16. Backup / restore certification

- CloudNativePG full backup + WAL/PITR.
- OpenBao Raft snapshot restore.
- Velero/Kubernetes objects et volumes pertinents.
- etcd snapshot restore lorsque necessaire.
- Git immutable backup restore.
- Object storage restore/versioning/immutability.
- Data platform rebuild/replay depuis Kafka/sources.

Un backup qui n'a jamais été restauré n'est pas considéré valide.

17. Data reconciliation

Après failover/failback/restore, vérifier la cohérence entre PostgreSQL, Kafka, OpenSearch, ClickHouse et données dérivées.

```
counts
aggregate versions
offsets/checkpoints
checksums by batch
business invariants
sampling
rebuild derived data if needed
```

Les projections dérivées peuvent être reconstruites automatiquement ; les sources critiques ne sont pas corrigées automatiquement sans règle métier sûre.

18. PostgreSQL production readiness

Vérifier failover, PITR, WAL, autovacuum, bloat, pg_stat_statements, connection pools PgBouncer, long transactions, replication lag et storage latency.

Exécuter des requêtes critiques sous charge et valider les plans/indexes.

19. Kafka readiness

Vérifier RF=3, min ISR=2, partitions, throughput, consumer lag, broker loss, MirrorMaker2 inter-site, schema registry, idempotent consumers et replay.

20. RabbitMQ readiness

Vérifier 3 nodes/site, Quorum Queues, retry/DLQ, queue pressure, node loss, consumer scale KEDA et reprise des jobs critiques.

21. Redis readiness

Vérifier Redis Cluster, slots, node loss, reconstruction des caches/paniers à partir de sources durables et absence de source de vérité critique dans Redis.

22. Search / OpenSearch readiness

Vérifier Search p95, indexing lag, reindex zero-downtime, mapping/template versioning, reconstruction des projections et comportement en mode dégradé.

23. Security final gates

- Keycloak MFA/Passkey, token rotation/revocation, RBAC/permissions.
- OPA/Kyverno politiques actives.
- OpenBao dynamic secrets et audit.
- SPIFFE/SPIRE workload identity.
- Kubernetes encryption-at-rest/KMS rotation testée.
- Cilium default-deny + egress allowlists.
- Squid deny-by-default et audit.
- Webhook signature/anti-replay/idempotence.
- PAN/CVV absents des stores/logs/traces.
- Trivy/Semgrep/gosec/Gitleaks/ZAP gates.

- SBOM + Cosign + Harbor digest pinning.
- Break-glass teste et audite.

24. Supply-chain release evidence

```
release_version
commit_sha
image_digest
SBOM digest
signature status
CVE scan result
license policy result
contract tests
E2E result
performance evidence
security evidence
approvals
FluxCD deployment status
```

25. Observability readiness

Verifier Prometheus/Thanos, Grafana, Loki, Tempo, Hubble, Alertmanager et Splunk security pipeline.

Executer des synthetic incidents et verifier qu'une alerte arrive avec owner, severity, runbook et contexte suffisant.

26. SRE/AIOps readiness

K8sGPT et KubeBuddy restent read-only. Tester qu'ils peuvent enrichir un incident sans modifier la production.

```
Alert -> context -> K8sGPT/KubeBuddy diagnosis
-> suggested remediation
-> PR
-> CI/policies
-> FluxCD
```

27. Automated runbooks

Certifier les runbooks SAFE par tests/Chaos. Les runbooks APPROVAL/MANUAL doivent avoir preconditions, rollback, verification et escalation.

28. Incident management readiness

Simuler au minimum un SEV1 et un SEV2. Verifier Incident Commander, communications, timeline, handover, postmortem et corrective actions.

29. On-call readiness

Verifier rota/astreinte, acces Zero Trust/PAM, credentials temporaires, runbooks et acces aux dashboards. Aucun ingenieur d'astreinte ne doit dependre d'un secret personnel non partage de facon securisee.

30. Backstage readiness

Chaque service critique doit exposer owner, Tier, SLO, runbook, dashboards, dependencies, rollback, backup/restore, PRA et scorecard production-ready.

31. Change management

Verifier PR obligatoire, CI green, protected branches, CODEOWNERS, double approval Tier 0, change_id, freeze policy et emergency path break-glass avec reconciliation Git obligatoire.

32. Migration readiness

Tester expand -> migrate -> contract, N/N+1 compatibility, backfills idempotents/reprenables et rollout Site B -> observation -> Site A.

33. External provider resilience

Tester PSP, carriers, Postal/Jasmin/Novu et autres adapters : timeouts, retry budgets, circuit breakers, reconciliation et modes dégradés.

34. Privacy readiness

Tester data minimization, access/export, erasure/anonymisation, retention-as-code, redaction logs et propagation aux caches/projections.

35. Data platform readiness

Vérifier Debezium/Kafka Connect, ClickHouse, dbt tests, DataHub lineage, Superset read-only et replay/reconstruction sans charge analytique sur OLTP.

36. Cost / capacity evidence

Documenter coût infra mensuel, utilisation moyenne/pic, headroom et seuil de scale-out. La phase initiale reste OSS/self-hosted first ; la documentation doit conserver les interfaces portables pour une migration managed future.

37. Production readiness dashboard

```
PRODUCTION READINESS
Application PASS
SLO PASS
Performance PASS
Capacity PASS
Security PASS
Backups PASS
Restore PASS
Failover PASS
Failback PASS
PRA rebuild PASS
Data integrity PASS
Observability PASS
Runbooks PASS
On-call PASS
Compliance tech PASS
```

Decision: GO / CONDITIONAL GO / NO-GO

38. Blocking criteria

- Split-brain possible ou fencing non prouvé.
- Payment/Order idempotence non prouvée.
- PITR/restore critique échoué.
- Backups non accessibles en incident.
- SLO critique rate sous charge cible.
- CVE/sécurité critique non acceptée avec compensating control valide.
- Secrets/PAN/CVV exposés.
- Failover Site A/B non fiable.
- Data corruption/reconciliation fail critique.
- Absence de runbook/owner pour un Tier 0.

39. Conditional GO policy

Un CONDITIONAL GO n'est permis que pour des écarts non critiques, avec owner, risque, compensating control, deadline et plan de vérification.

```
finding_id
severity
risk
compensating_control
owner
due_date
verification_plan
approval
```

40. Evidence package final

Produire un dossier de certification rattache a la release :

```
production-readiness/
summary.md
go-no-go.json
slo/
k6/
playwright/
security/
chaos/
backups/
restore/
failover/
failback/
pra/
data-reconciliation/
capacity/
approvals/
runbooks/
incidents/
```

41. Ordre d'execution

```
1 Freeze candidate release
2 Validate supply-chain evidence
3 Deploy certification environment
4 Functional/E2E smoke
5 Contract/integration gates
6 Security gates
7 Load/performance/soak
8 Capacity analysis
9 Chaos tests
10 Backup/restore tests
11 Failover Site A -> B
12 Failback B -> A
13 Full PRA rebuild
14 Data reconciliation
15 Incident/on-call drill
16 Privacy/compliance technical tests
17 Collect evidence
18 Review blockers
19 GO / CONDITIONAL GO / NO-GO
```

42. Definition of Done

- Tous les parcours critiques ont des preuves E2E et backend.
- Les SLO et objectifs de latence tiennent sous charge cible.
- La capacite conserve une marge acceptable au peak.
- Les pannes majeures sont testees avec recovery observe.
- PostgreSQL PITR, OpenBao, Git et autres backups critiques sont restaurables.
- Failover et failback multi-site sont executes sans split-brain.
- Un site peut etre reconstruit depuis zero via PRA-as-Code.

- La reconciliation des donnees apres recovery est PASS.
- Les controles security/privacy/supply-chain sont PASS.
- Observability, alerting, runbooks et astreinte sont operationnels.
- Backstage expose un statut production-ready exploitable.
- Le dossier de preuves est complet et rattache a la release.
- Une decision GO/CONDITIONAL GO/NO-GO est justifiee par des preuves.

La regle finale est simple : pas de GO sur intuition. Pas de GO parce que 'tout semble vert'. Le GO exige des preuves de performance, securite, restauration, failover, integrite data et capacite operationnelle.